

Compte rendu Projet cowsay

Dans le cadre de notre projet encadré, nous nous sommes concentrés sur l'exploration et l'exploitation des fonctionnalités offertes par la commande cowsay.

Partie Bash :

Au début de la partie Bash, nous avons effectué une étude approfondie de la commande cowsay, en examinant ses fonctionnalités et ses options disponibles que nous avons regroupées dans un tableau de référence. Par la suite, nous avons élaboré plusieurs scripts mettant en scène une vache affichant de manière dynamique les itérations d'une suite de nombres. Cette mise en œuvre a impliqué l'utilisation des fonctions sleep, clear et cowsay.

Pour notre script "crazy_cow", nous avons opté pour l'implémentation de la suite de Syracuse. La suite de Syracuse est une suite de nombre définie de la façon suivante : On part d'un nombre entier U_0 non nul, S'il est pair, on le divise par 2 et s'il est impair, on le multiplie par 3 et on ajoute 1. Et on recommence l'opération avec le résultat. On obtient ainsi une suite de nombres. Ce qui est remarquable avec cette suite, c'est que chaque fois qu'on a essayé avec un nombre, on est tombé sur 1 au bout d'un certain nombre d'essais mais personne n'a pour l'instant réussi à démontrer que c'était vrai pour tous les entiers. Cette suite de nombres, initiée avec le premier argument passé dans la ligne de commande, est affichée à l'aide de la vache. En option, le script accepte un second argument qui spécifie la durée de la pause entre chaque affichage. Par défaut, cette durée est fixée à 0.8 seconde.

Partie C :

Dans la partie C, nous avons recoder la routine cowsay afin de lui conférer de nouvelles fonctionnalités et une plus grande flexibilité.

Le code source de "newcow.c" représente une modification de la commande cowsay. Cette version améliorée permet de personnaliser différents aspects de l'apparence et du comportement de la vache. Parmi les options désormais disponibles, nous retrouvons :

- --eye ou -e : qui prend une chaîne de 2 caractère pour modifier le dessin des yeux de la vache.
- --tongue ou -T : qui prend un caractère pour spécifier la langue utilisée par la vache.
- --tail : qui prend un entier qui permet d'ajuster la longueur de la queue de la vache.
- --troupe : qui prend un entier, puis qui affiche un troupeau de vaches correspondant à l'entier
- --text : qui prend une chaîne de caractère pour permettre à la vache de s'exprimer en disant cette chaîne de caractères.
- --think : qui prend une chaîne de 2 caractère pour que la vache médite sur cette phrase donnée.
- --broute : pour simuler le comportement de broutage d'herbe par la vache.
- --milk : pour obtenir du lait de la vache.

Le code source "wildcow.c" illustre une approche plus amusante avec une animation mettant en scène une vache effectuant une acrobatie. Cette animation utilise la fonction update() pour rafraîchir l'écran et la fonction goto(x, y) pour se placer aux coordonnées x,y.

Enfin, le script "reading_cow.c" permet à la vache de lire caractère par caractère un fichier texte passé en argument en ligne de commande, ou bien l'entrée standard stdin si aucun fichier n'est spécifié.

Partie Automates :

Dans la partie automate nous avons tout d'abord imaginé et dessiné plusieurs automates afin de nous aider à implémenter les règles nécessaire à la création du Tamagoshi. On retrouve quelques options de la partie précédente afin de personnaliser la vache. Nous devons donc créer un Tamagoshi qui consiste à nourrir la vache pour qu'elle vive le plus longtemps possible, en jouant avec des quantités de nourriture qui varient aléatoirement et un niveau de santé invisible qui varie avec la nourriture donnée (aléatoire aussi avec la digestion) si ce n'est avec l'affichage de la vache, mais qui ne permet pas de déterminer la santé précisément. Après pas mal de tests le jeu fonctionne très bien, il suffit d'exécuter le programme et on peut jouer, notre record est pour l'instant de 28 itérations de la boucle.

En conclusion, ce projet nous a non seulement permis d'appliquer les connaissances acquises en cours, mais également de découvrir et de mettre en pratique de nouveaux concepts. Ce projet a été une belle occasion pour nous d'explorer de nouvelles idées, d'apprendre et de nous amuser.

Options de la commande cowsay:

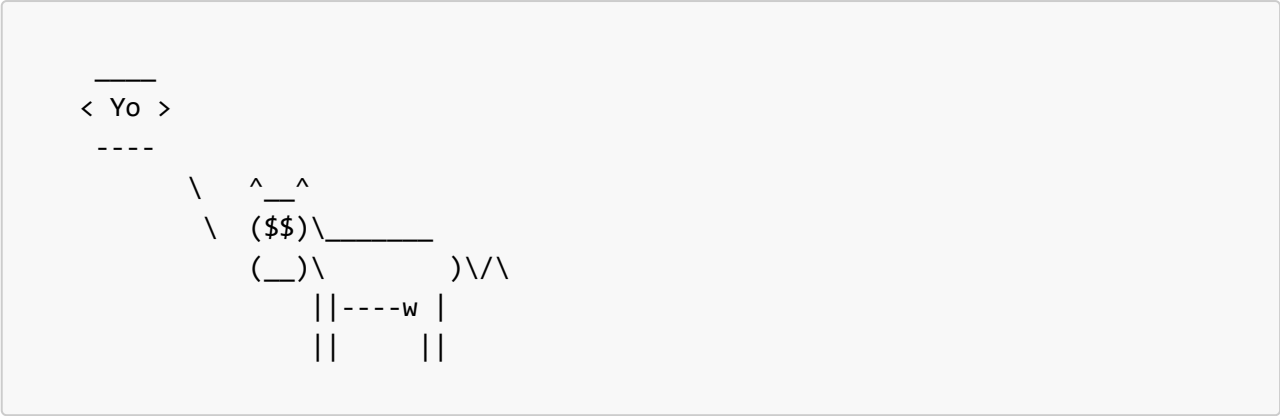
- -n : Désactive le retour à la ligne
- -w Spécifie la largeur de la bulle en colonnes. La valeur par défaut est 40.
- -b : « Mode Borg », utilise == à la place de oo pour les yeux de vache.



- -d : "Dead", utilise X



- -g : "Gourmand", utilise \$\$



- -p : "Paranoïaque, utilise @@



- -s : "Stoned", utilise ** pour représenter des yeux injectés de sang , plus un U descendant pour représenter une langue extrudée.



- -t : "Fatigué", utilise --



- -w : "Wired", utilise OO

```

  _____
< Yo >
  -----
           \   ^__^
            \  (oo)\_______
               (__)\       )\/\
                 ||----w |
                 ||     ||

```

- -y : "Jeune", utilise ..

```

  _____
< Yo >
  -----
           \   ^__^
            \  (..)_______
               (__)\       )\/\
                 ||----w |
                 ||     ||

```

- -e : Spécifie manuellement le type d'œil de la vache
- -T : Spécifie manuellement la forme de la langue de la vache
- -f : Spécifie un type d'art de vache.

cowsay -f eyes "I see you"

```

< I see you >
-----
  \
  \
                                     .:!!!!!!!:.
.:!!!!!:.                          .:!!!!!!!
~~~~!!!!!!                          .:!!!!!!!UWW$$$
:$NWX!!:                            .:!!!!!!XUWW$$$$$$$P
$$$$$#WX!:.                        .<!!!!UW$$$"  $$$$$$#
$$$$$  $$$UX  :!!UW$$$$$$$  4$$$$$*
^$$$B  $$$$\  $$$$$$$$$$  d$$$R"
  "$bd$$$  "$$$$$$$$$$o+#"
      """"      """"""

```

- -l : Répertoire les noms des types d'art de vache disponibles dans le répertoire COWPATH

Kikabay Burhanuddin

Gailleton Lucas

IMA-4